

Processes

Design of Telematic Systems

Second Course. Bachelor in Robotics Engineering
Universidad Carlos III de Madrid
2026–2027

Instructions:

- The assignment must be done in groups of 2 or 3 students.
- Your code must be well documented. You must include comments in your code to explain the different parts of your code.
- There should be clear instructions on how to compile and run your code. Include a README file with the instructions within your submission.
- You must submit the source code of the exercises, a README file and a report in PDF format.
- You must submit your assignment through Aula Global.
- Remember that you will be using your own code on the individual lab exam.
- The grading will be proportional to the percentage of assignment implemented.

Objective

The objective of this graded lab is to implement a game where adventurers navigate through a dungeon maze under a Fog of War regime, discovering the content of each cell only as they explore it. In subsequent steps, adventurers will collaborate within a party coordinated from outside the maze by a Mentor, who will collect their discoveries via IPC, reconstruct the dungeon map, and issue movement orders. Finally, the dungeon will be populated by autonomous monsters, such as goblins and orcs (*Gosh darn it, Donut!*), requiring the Mentor to safely guide all party members to the exit stairwell while keeping them alive.

Understanding additional files

The Dungeon Maze

The maze is represented by a grid of squares. Each square or cell can contain a surprise/prize box or an obstacle (e.g., a wall) that makes impossible to access to that square. Obstacles and treasures do not occupy the same cell. This information is not directly accesible by the adventurer or the mentor.

The information of the maze is stored in a file with the following structure:

- First line: number of rows, number of columns, position of the exit stairwell.
- Second line: number of obstacles and position of each obstacle.
- Third line: number of boxes and position of each box.

We assume that obstacles and boxes do not occupy the same cell.

Example:

```
6 10
3 (0,0) (1,2) (3,4)
2 (1,1) (2,3)
```

This dungeon maze is 6x10, and has 3 obstacles located at (0,0), (1,2), (3,4) and 2 boxes at (1,1) and (2,3).

Neither the mentor nor the adventurers are allowed to read directly this file. This information will be given by the `class GogglesSpell`.

You can find two examples of this file at Aula Global, named `level_one.txt` and `level_two.txt`. Remember that your code should work for any given file name.

Position of the adventurers within the dungeon

The mentor (to be implemented in the second part) needs to know the number of adventurers of his/her/their party and their position within the room. This information is stored in a file that has one line per adventurer. Each line has:

- A pair of integers representing the position of the adventurer.
- Initial Health.
- Initial Mana Points: this allows the adventurer to cast spells, e.g. to cast the `whatsthat` spell of the `class GogglesSpell`.
- Initial Gold.
- Name of the Adventurer.

Example:

```
(2,3) 100 50 50 Carl  
(2,2) 100 100 70 Princess Donut  
(1,4) 100 0 50 Kitiara  
(1,5) 60 150 20 Raistlin
```

This file states that in this party of adventurers there are 4 people, in the initial positions (2,3), (2,2), (1,4) and (1,5). For example, Princess Donut has 100 health points, 100 mana points and 70 gold coins.

You can find two examples of this file at Aula Global, named `party.txt` and `other_party.txt`. Remember that your code should work for any given file name.

Position of the monsters within the dungeon

The Dungeon Maze will be populated by monsters, such as goblins and orcs (as said before: *Gosh darn it, Donut!*) (part 3 of the assignment). These monsters will be placed in the dungeon at specific positions, and the adventurers will crash into them if they try to move into a cell occupied by a monster.

This information will be stored in a file that has one line per monster. Each line has:

- A pair of integers representing the position of the monster.
- Initial Health.
- Name of the Monster.

Example:

```
(1,1) 40 The Grey Gentlemen  
(4,4) 50 Steve the Troll  
(2,1) 100 Detritus  
(1,3) 100 Smaug
```

This file states that in this dungeon there are 4 monsters, in the initial positions (1,1), (4,4), (2,1) and (1,3). For example, Smaug has 100 health points.

You can find two examples of this file at Aula Global, named `monsters.txt` and `monsters_2.txt`. Remember that your code should work for any given file name.

The magical goggles

Each adventurer will have a set of magical goggles that will use to know if a given position has an obstacle (making impossible to move), or a surprise box.

The code of the magical goggles is already provided and you can find it at Aula Global, under the name `goggles.py`.

Working with the GogglesSpell class

You should use the `class GogglesSpell`, by importing the class, and you will interact with it only through its methods. **You are not allowed to access to any internal variable.**

In order to be able to work with the `class GogglesSpell`, you should copy the Aula Global file `goggles.py` in your work directory and include this line at the beginning of your code:

```
import goggles
```

The `class GogglesSpell` exposes the following methods:

- `print_dungeon()`: help function for debugging. It prints the dungeon. It is only for testing purposes and should not be used in your submission.
- `dimensions()`: returns the dimensions of the room. It can be used only by the mentor.
- `where_is_the_exit()`: returns the position of the exit. It can be used only by the mentor at the beginning of the game.
- `what_is_here(row, column)`: returns the type of cell in the given position. The possible values are:
 - `'-'`: empty cell.
 - `'E'`: exit stairwell.
 - `'B'`: surprise box.
 - `'#'`: obstacle. The adventurer cannot move into this cell.

- `type_of_box(row, column)`: returns the type of the box in the given position. The possible values are:
 - `'MP'`: mana potion. Augments the mana of the adventurer by 20 points.
 - `'MD'`: mana drain. Reduces the mana of the adventurer by 10 points.
 - `'HP'`: health potion. Augments the health of the adventurer by 20 points.
 - `'HD'`: health drain. Reduces the health of the adventurer by 10 points.
 - `'G'`: gold. Augments the gold of the adventurer by 10 coins.

You can find at Aula Global `test.py`, an example on how to use the `GogglesSpell` class.

```
$ cat level_one.txt
6 10
(2,2)
3 (0,0) (1,2) (3,4)
2 (1,1) (2,3)
$ python3 test.py level_one.txt
# - - - - -
- B # - - - - -
- - E B - - - - -
- - - - # - - - - -
- - - - - - - - -
- - - - - - - - -

dimensions of the dungeon: (6, 10)
exit position: (2, 2)
what is here at (2, 3): B
type of box at (2, 3): HD
$ cat level_two.txt
3 15
(2,10)
5 (1,0) (1,2) (2,13) (1,5) (0,14)
3 (0,0) (2,7) (1,9)
$ python3 test.py level_two.txt
B - - - - - - - - - #
# - # - - # - - - B - - - - -
- - - - - - B - - E - - # -
```

```
dimensions of the dungeon: (3, 15)
exit position: (2, 10)
what is here at (2, 3): -
$
```

Exercise 1. An adventurer in a dungeon maze

Implement a program that simulates the adventurer going through a dungeon maze. The dungeon is a grid of squares and the adventurer moves in the dungeon one square at a time. It can move up, down, left or right. However, it cannot move out of the room, or into a cell with an obstacle.

You **should use** the interface provided by the provided `GogglesSpell` class. You are not allowed to change the provided code or to access the room file directly.

Each time the adventurer invokes the method `what_is_here(row, column)` of the `GogglesSpell` class, she will discover the content of the cell in the given position and lose 2 points of mana (as this function is a spell). If it is a surprise box, she can invoke the method `type_of_box(row, column)` to know its content, with no charge.

The adventurer is controlled by a user through a command-line interface. The program admits the following commands:

- `mv <direction>`: tries to move the adventurer one cell in the specified direction. (up, down, left, right). If it is possible, the adventurer changes its position and prints `OK`. If it is not possible to move in the specified direction, its position does not change and prints `KO`. The adventurer cannot move if her health is 0 or if she has not enough mana to cast the spell `what_is_here(row, column)`.
- `health`: prints the current health of the adventurer.
- `pos`: prints the current position of the adventurer as a 2-dimension vector.
- `unbox`: if there is a box in the current position, it prints the content of the box and the adventurer's health, mana and gold are updated accordingly. If there is no box in the current position, it prints `No box`.
- `exit`: the program prints the current position of the adventurer, her health, mana and gold, and exits.

The program should handle these signals:

- SIGUSR1: if the adventurer has enough gold, she can buy a mana potion. The adventurer's mana is increased by 10 and her gold is decreased by 5. If it does not have enough gold, it prints `Not enough gold`.
- SIGUSR2: if the adventurer has enough mana, she can cast a spell to increase her health (even if the health is 0). The adventurer's health is increased by 10 and her mana is decreased by 5.
- SIGQUIT: the program prints the current position of the adventurer, her health, mana and gold.
- SIGTSTP: the adventurer is *petrified*. This signal should be caught and managed. The adventurer should print its identifier, position and health each time it receives this signal. And it should not respond to any command until it is *depetrified* by the signal SIGCONT (the only one that should be handled while the adventurer is *petrified*). During petrification, the time does not pass for the adventurer, so her health does not decrease. If the adventurer is already *petrified*, it should ignore this signal.
- SIGCONT: the adventurer is *depetrified*. If the adventurer is not *petrified*, this signal should be ignored.
- SIGINT: the health of the adventurer is decreased by 10. If the health is 0, the adventurer cannot move.

Additionally, you should *simulate* the passing of time the health of the adventurer by 1 point each 2 seconds. You should use the signal SIGALRM to do this. Remember that during petrification, the time does not pass for the adventurer, so her health does not decrease if she is *petrified*.

The adventurer program receives the following arguments (**in any order**):

- A number that identifies the adventurer.
- `-f <filename>:-` the name of the file that contains the dungeon information.
- `-pos <row> <column>:` (optional) the initial position of the adventurer. If not given, the position is `(0, 0)`.
- `-h <health>:` (optional) the initial health of the adventurer. If not given, the health is 100.
- `-m <mana>:` (optional) the initial mana of the adventurer. If not given, the mana is 50.
- `-g <gold>:` (optional) the initial gold of the adventurer. If not given, the gold is 25.
- `-n <name>:` (optional) the initial name of the adventurer. If not given, the name is `Princess Donut`.

Example:

```
$ python3 adventurer.py 3 -pos 2 3 -h 80 -m 50 -g 25 -n Arya -f level_one.txt
```

Here we are invoking a adventurer named Arya with identifier 3, her initial position is row 2, column 3, her initial health is 80, her initial mana is 50 and her initial gold is 25. The dungeon information is in the file level_one.txt.

At the very beginning the adventurer should print its name and PID to standard error and create an instance of the `GogglesSpell` class with the filename given as input. Then, it should create if the adventurer's initial position is valid (if not, prints `Invalid initial position` to standard error and exits) and waits for the user commands. For this first cast of the spell, the adventurer should not lose mana points. The following calls to the spell `what_is_here(row, column)` will decrease the adventurer's mana by 2 points.

All error messages should be printed to the standard error stream.

Execution examples

We will use as example the output of the Python implementation. All these examples use as dungeon description file the following.

```
6 10
(2, 2)
3 (0, 0) (1, 2) (3, 4)
2 (1, 1) (2, 3)
```

```
$ python3 adventurer.py 3 -pos 0 0 -f level_one.txt
My name is Princess Donut, my PID is 12099 and my id is 3
(0, 0) Invalid initial position
```

```
$ python3 adventurer.py 3 -pos 2 3 -h 80 -f level_one.txt -n Arya
My name is Arya, my PID is 12108 and my id is 3
PID: 12108
health
Health: 78
pos
Position: 2 3
unbox
Type: HD
Health: 68 Mana: 50 Gold: 25
```



```
mv up
OK
mv up
OK
mv up
Arya: I cannot move up
KO
pos
Position: 0 3
health
Health: 9
health
Health: 5
health
Health: 0
mv down
Arya: I cannot move down
KO
mv left
Arya: I cannot move left
KO
pos
Position: 0 3
exit
Arya: Position: 0 3 Health: 0 Mana: 44 Gold: 25
```

In the following example we send the signal SIGTSTP (using CTRL+Z), we try several commands (without success as the adventurer is *freeze*), we *resume* the execution by sending SIGCONT from another terminal (using `kill -s CONT 6635`) and then, we send SIGQUIT (using CTRL+\) to print the position and points of the adventurer. Finally, we send to the process SIGINT (using CTRL+C) to decrease the health of the adventurer.

```
$ python3 adventurer.py 3 -pos 3 3 -h 50 -f level_one.txt
My name is Princess Donut, my PID is 6635 and my id is 3
health
Health: 48
pos
Position: 3 3
^Zpos
Princess Donut is petrified
health
Princess Donut is petrified
unbox
```

```
Princess Donut is petrified
SIGCONT received. Princess Donut is no longer petrified
pos
Position: 3 3
health
Health: 46
^\\
Princess Donut: Position: 3 3 Health: 44 Mana: 50 Gold: 25
^\\
Princess Donut: Position: 3 3 Health: 42 Mana: 50 Gold: 25
unbox
No box at 3 3
^C
Princess Donut: Position: 3 3 Health: 30 Mana: 50 Gold: 25
exit
Princess Donut: Position: 3 3 Health: 28 Mana: 50 Gold: 25
```

Information to add to your report

Describe the behaviour of your program drawing by hand (and then scanning) the following diagrams:

1. A class diagram of the classes you have implemented, with the functions they provide.
2. A flowchart or an activity diagram of the main loop of the adventurer. It should show the flow of execution and the decision points. Additionally, it should include the handling of signals and the interaction with the user.

As you are going to hand draw the diagrams, ensure that they are clear and legible.

Exercise 2. The Mentor of the adventurers

This program will be the mentor of a party of adventurers. It will control the adventurers, sending them commands to move through the dungeon maze. The program admits the following commands:

- `mv <id/all> <direction>`: moves the adventurer in the specified direction. (up, down, left, right).
 - `id` is the identifier of the adventurer. If `id` is *all*, the command is sent to all adventurers.
 - If it is not possible to move in the specified direction (because there is a wall, an obstacle, another adventurer in the desired new place or the adventurer has no enough health), the adventurer does not move.
 - When `id` is *all*, the movement should be tried in order, e.g. first the adventurer with identifier 1, then the adventurer with identifier 2, and so on.
 - If the adventurer finds a surprise box, prints `Surprise box found by adventurer X, NAME!`, being `X` the identifier of the adventurer and `NAME` the name of the adventurer.
 - If the desired movement makes two adventurers collide, the adventurer does not move and prints `Collision between adventurers x and y`, being `x` and `y` the identifiers of the adventurers.
- `print`: prints the information the mentor has so far.
- `pos <id/all>`: prints the current position of the adventurer identified by `id`. If `id` is *all*, the command is sent to all adventurers.
- `points <adventurer_id/all>`: prints the current points of the adventurer identified by `id`. If `id` is *all*, the command is sent to all adventurers.
- `petrify <id/all>`: petrifies the adventurer identified by `id`. If `id` is *all*, petrifies all adventurers.
- `depetrify <id/all>`: resumes the adventurer identified by `id`. If `id` is *all*, depetrifies all adventurers.
- `exit`: finishes the program. It sends the command to all the adventurers, printing the information sent by them: the position and the battery level. It also prints the position of all found treasures.

Two adventurers cannot occupy the same cell unless it is the exit stairwell. When all the adventurers in the party are in the exit stairwell, the program prints

All adventurers have reached the exit stairwell! and finishes.

The program should also have track of the amount of total gold of the party. The program admits the following signals:

- `SIGINT`: it finishes the program. It prints the current position of the adventurers, their information (name, health, mana, gold), waits for the adventurers to die and exits.
- `SIGTSTP`: it decreases the health of all the adventurers by 10.
- `SIGUSR1`: sends to all the adventurers the signal to buy, if they have enough gold, a mana potion. It prints the information sent by the adventurers: the position, their points and updates the total gold of the party.
- `SIGUSR2`: sends to all the adventurers the signal to cast a spell to increase their health, if they have enough mana. It prints the information sent by the adventurers: the position and their points.
- `SIGQUIT`: prints the current position and battery level of all the adventurers.

Additionally:

- If there are n adventurers, the program will create n children, one for each adventurer.
- Each adventurer will be controlled by the program of the previous exercise (`adventurer.py`).
- The program will use pipes to communicate with the adventurers.
- When finishing the program must print the exit status of its children.
- It should store the position of the adventurers and the information gathered so far about the dungeon.
- It should NOT store the specific health, mana or gold of the adventurers. This information should be requested to the adventurers when needed.
- It should have track of the amount of total gold of the party.
- During the execution, if a movement leads to a collision between adventurers, it should warn the user and not allow the adventurers to move.
- It should also check if the initial position of the adventurer is not another adventurer's position, a wall or an obstacle. If so, the program must print an error message and finish.
- Only at the beginning of its execution the mentor is allowed to use an instance of the `GogglesSpell` class. It can use its functions to know the dimensions of the room, and to check if the **initial** position of the adventurers are

valid. Once the setup is finished, the mentor cannot use the sensor functions again.

The mentor receives the following arguments (**in any order**):

- `<string>`: the name of the mentor. If not given, the name is Gandalf.
- `-dungeon <filename>`: the name of the file that contains the dungeon information.
- `-party <filename>`: the name of the file that contains the adventurers information.

Example:

```
$ python3 mentor.py -dungeon dungeon.txt -party party.txt
```

Information to add to your report

Describe the behaviour of your program drawing by hand (and then scanning) the following diagrams:

1. A class diagram of the classes you have implemented, with the functions they provide.
2. A flowchart or an activity diagram of the main loop of the mentor. It should show the flow of execution and the decision points and the interaction with the adventurers within the party. Additionally, it should include the handling of signals and the interaction with the user.

Exercise 3. The Dungeon Master and the monsters

You should implement a program that simulates a Dungeon Master. This Dungeon Master will know the position of all the entities in the dungeon (monsters and adventurers) and will control the health of the monsters, making them disappear when their health is 0.

For this, the DM will receive in a file the position of the monsters and their initial health (see section *Position of the monsters within the dungeon*). The DM will also receive the name of the dungeon description file (see section *The dungeon maze*), so the DM can check if the initial position of the monsters is valid (not an obstacle, a wall or another monster). The monsters are static, i.e. they do not move, but they can fight with the adventurers if they are in the same cell.

The DM will communicate with the mentor through a FIFO:

- Each time there is a change in an adventurer's position, the mentor will send a message to the DM with the new position of the adventurer.
- The DM will check if there is a monster in that cell. If there is, it will decrease the health of the monster by 10 points.
- If the monster reaches 0 health or less, the dungeon master will send a message to the mentor to inform that the monster has been defeated and it will remove the monster from its list of monsters. The adventurer will not decrease its health if the monster is defeated.
- If the monster is still alive, the DM will send back to the mentor a message to inform that there is a monster in that cell. Then, the mentor will send a signal to the adventurer to decrease its health by 10 points.

The DM should also know when the mentor finishes the game, so it can finish its execution. For this, the mentor can send a message to the DM when the game is finished.

You should design a protocol to communicate between the mentor and the dungeon master through the FIFO and throughout the game. You can use any format you want, but it should be clear and easy to parse. You should also design the messages printed by the mentor and the dungeon master to inform the user about the events happening in the dungeon.

Information to add to your report

Describe the behaviour of your program drawing by hand (and then scanning) the following diagrams:

1. A description of the protocol you have designed to communicate between the mentor and the dungeon master. It should include the format of the messages, the possible commands and the expected responses. This should be a sequence diagram that shows the flow of messages between the mentor and the dungeon master. You can include a table with the possible messages and their format, to make it easier to understand the protocol.
2. A flowchart or an activity diagram of the main loop of the DM. It should show the flow of execution and the decision points and the interaction with the mentor. Additionally, it should include the handling of signals and the interaction with the user.
3. Enumerate the changes implemented on the code of the mentor to include the communication with the DM.